Diseño de algoritmo con estructuras anidadas en Java

Proyecto: ClasificadorUsuarios

Curso: Fundamentos de Programación

Estudiante: Lizney Bolaño

Corporación Universitaria Americana

Ingeniería Industrial, modalidad virtual

Noviembre de 2026

Índice

1. Introducción

2. Justificación teórica

3. Diseño algorítmico

4. Implementación

5. Resultados de las pruebas

6. Discusión

7. Recomendaciones de mejora

8. Conclusiones

Bibliografía

Anexo A. Casos de prueba

1. Introducción

En muchas aplicaciones digitales, como plataformas de video, bibliotecas en línea o servicios de música, no todos los usuarios ven el mismo contenido. Lo que una persona puede usar depende de su edad y del tipo de cuenta que tiene. Por ejemplo, un niño con una cuenta gratuita no debería tener el mismo acceso que un adulto con una cuenta premium. Por esta razón, el sistema necesita un módulo que clasifique a cada usuario antes de mostrarle el contenido.

El objetivo de este reto fue diseñar un algoritmo que use condicionales anidadas y traducirlo a Java dentro de un proyecto llamado ClasificadorUsuarios. El programa recibe dos datos: la edad del usuario y su tipo de suscripción (gratuita o premium). Con esos datos, devuelve la categoría de edad y el nivel de acceso que le corresponde. Si los datos no son válidos, el programa muestra un mensaje de error.

El reto forma parte del curso de Fundamentos de Programación. Su propósito no es solo que el programa funcione, sino que el estudiante aprenda a planear la solución antes de escribir código. Por eso el trabajo se hizo en etapas: primero se definieron los requisitos, después se escribió el pseudocódigo y el diagrama de flujo, luego se programó en Java y, por último, se probó cada caso. Este orden permitió detectar la lógica que faltaba antes de llegar a la computadora.

Este informe explica por qué se usaron estructuras anidadas, cómo se diseñó el algoritmo, cómo se programó, qué resultados se obtuvieron en las pruebas, qué problemas tiene la solución y qué mejoras se pueden hacer.

2. Justificación teórica

Una estructura de decisión permite que un programa elija entre varios caminos según una condición. La más básica es el if-else. Cuando una decisión depende de otra decisión, se usa una estructura anidada, es decir, un if dentro de otro if. En este caso, el resultado depende de dos variables a la vez: primero se revisa el tipo de suscripción y, dentro de cada caso, se revisa la edad.

Esta forma de programar es adecuada por tres razones. La primera es que refleja la forma en que una persona pensaría el problema: primero pregunta qué tipo de cuenta tiene y después qué edad tiene. La segunda es que permite validar los datos en orden: si la edad es negativa o la suscripción no existe, el programa se detiene antes de llegar a las reglas de acceso. La tercera razón es que cubre todas las combinaciones posibles de edad y suscripción, lo que da un comportamiento predecible.

Este tipo de decisión aparece con frecuencia en la vida cotidiana. Por ejemplo, un cine cobra la entrada según la edad y el día de la semana, y una tienda aplica descuentos según si el cliente es socio y cuántos años tiene. En todos esos casos, el precio final depende de más de

una condición. Aprender a escribir estas reglas con condicionales anidadas es un paso importante para programar sistemas reales.

Hay que tener en cuenta que una estructura anidada muy profunda puede volverse difícil de leer. Por eso, en el diseño se intentó mantener solo dos niveles de anidación, y en la sección de recomendaciones se propone una forma más limpia de organizar el código. Las estructuras anidadas son útiles aquí porque el problema tiene dos dimensiones, edad y suscripción, y ambas afectan el resultado final.

3. Diseño algorítmico

3.1 Requisitos funcionales

Primero se definieron las reglas del sistema. Las categorías de edad son las siguientes:

Categoría	Rango de edad
Niño	De 0 a 11 años
Adolescente	De 12 a 17 años
Adulto	De 18 a 59 años
Adulto mayor	60 años o más

Los tipos de suscripción son dos: gratuita y premium. Cada combinación de categoría y suscripción tiene un nivel de acceso distinto, como se ve en la implementación.

3.2 Pseudocódigo

```
INICIO
  LEER edad
  LEER suscripcion
  SI edad < 0 ENTONCES
    MOSTRAR "ERROR: edad no valida"
  SINO
    SI suscripcion = "gratuita" ENTONCES
      SI edad < 12 ENTONCES
        MOSTRAR "Nino - Acceso gratuito infantil"
      SINO
        SI edad < 18 ENTONCES
          MOSTRAR "Adolescente - Acceso gratuito basico"
        SINO
          SI edad < 60 ENTONCES
            MOSTRAR "Adulto - Acceso gratuito estandar"
          SINO
            MOSTRAR "Adulto mayor - Acceso gratuito preferencial"
          FIN SI
        FIN SI
      FIN SI
    SINO
      SI suscripcion = "premium" ENTONCES
        (misma estructura de edades, con acceso premium)
      SINO
        MOSTRAR "ERROR: suscripcion no valida"
      FIN SI
    FIN SI
```

```
  FIN SI
FIN
```

3.3 Diagrama de flujo

El diagrama de flujo sigue la misma lógica del pseudocódigo. Primero se revisa si la edad es negativa. Si lo es, se muestra el error y el programa termina. Si no lo es, se revisa si la suscripción es gratuita. Si sí, se entra a las cuatro preguntas de edad. Si no, se revisa si es premium. Si tampoco lo es, se muestra el error de suscripción. Al final, el programa termina. El diagrama completo aparece en la Figura 1. Ese mismo flujo se verificó en PSeInt, a partir del archivo ClasificadorUsuarios.psc incluido en el repositorio.

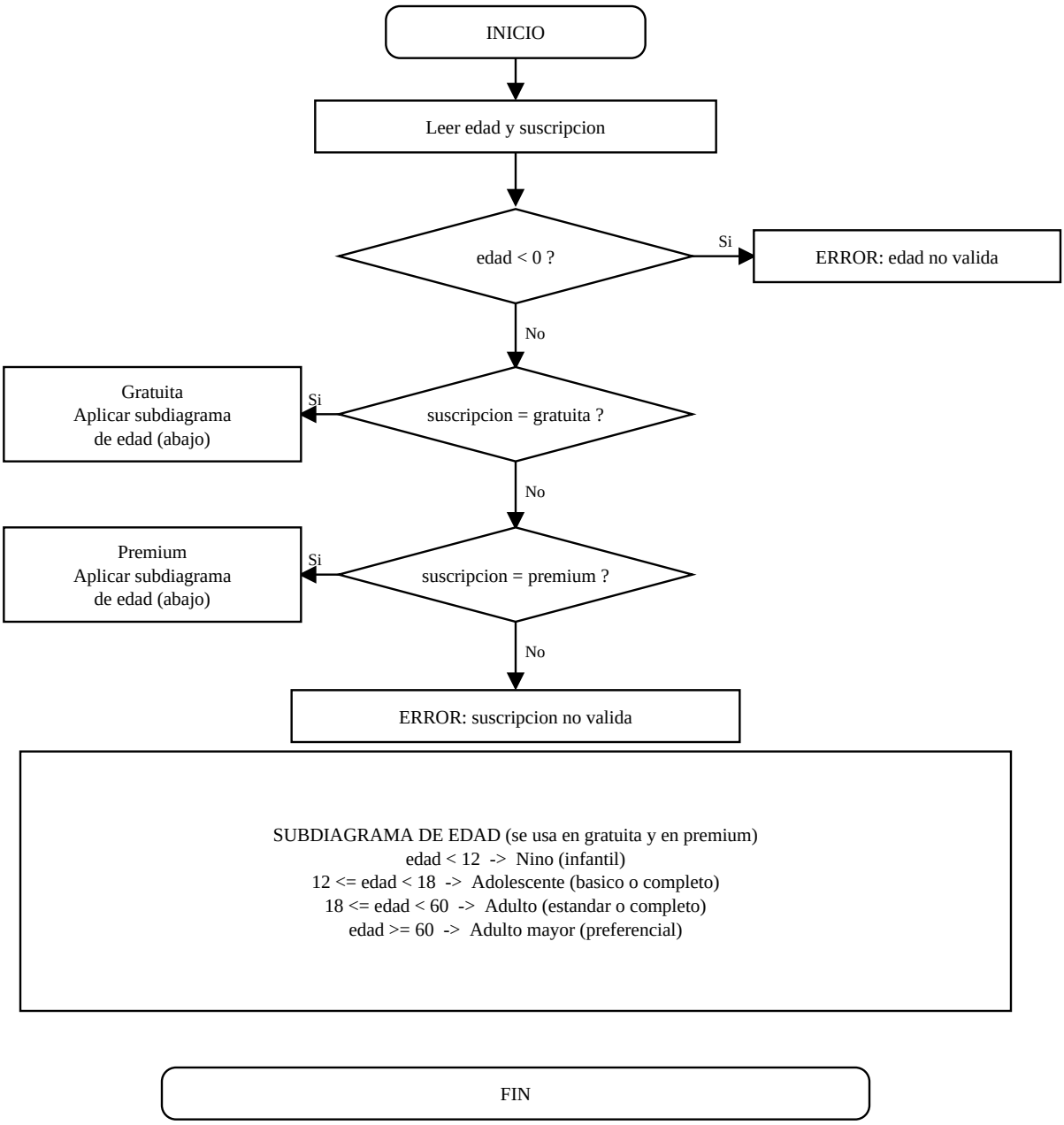


Figura 1. Diagrama de flujo del clasificador (dibujado con formas vectoriales).

Además del diagrama anterior, el mismo algoritmo se dibujó automáticamente en PSeInt, a partir del archivo de pseudocódigo. La Figura 2 muestra ese diagrama generado por el programa.

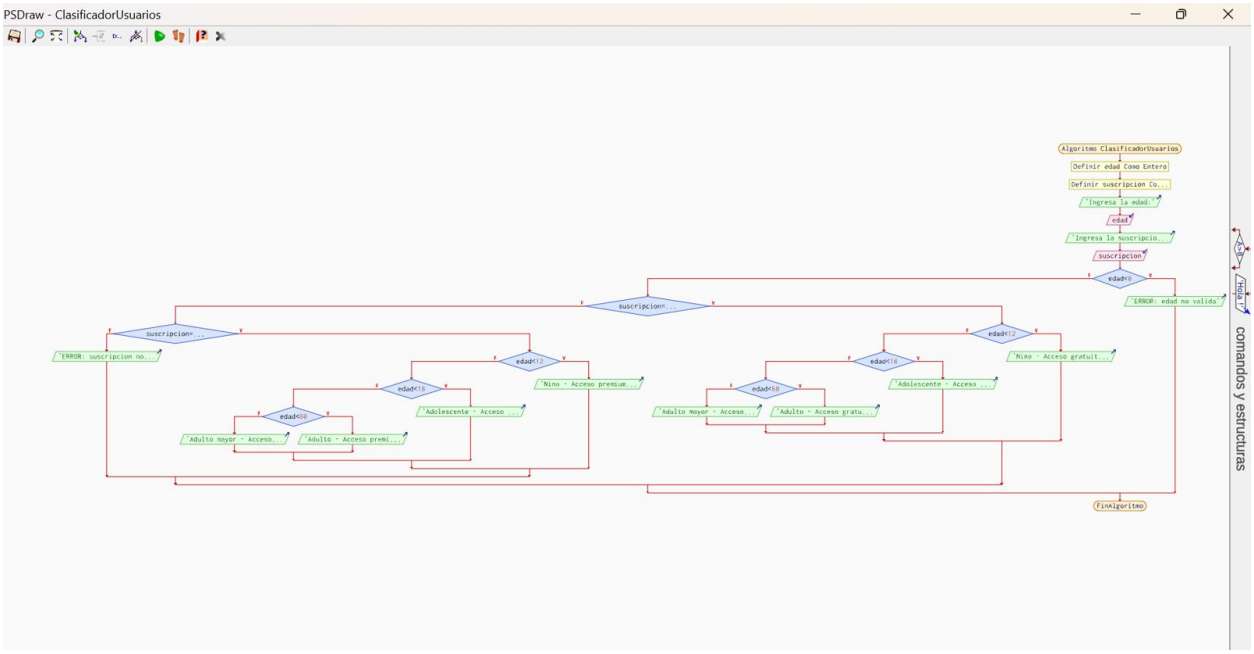


Figura 2. Diagrama de flujo generado en PSeInt (PSDraw) a partir de ClasificadorUsuarios.psc.

4. Implementación

El pseudocódigo se tradujo a Java dentro de la clase ClasificadorUsuarios. Se usó un método estático llamado clasificar, que recibe la edad como número entero y la suscripción como texto. Antes de comparar, el texto se limpia con trim() y se pasa a minúsculas, para que "Premium" o " gratuita " también se acepten.

```
public static String clasificar(int edad, String suscripcion) {
    if (edad < 0) {
        return "ERROR: edad no valida";
    }
    if (suscripcion == null) {
        return "ERROR: suscripcion no valida";
    }
    String tipo = suscripcion.trim().toLowerCase();
    if (tipo.equals("gratuita")) {
        if (edad < 12) {
            return "Nino - Acceso gratuito infantil";
        } else if (edad < 18) {
            return "Adolescente - Acceso gratuito basico";
        } else if (edad < 60) {
            return "Adulto - Acceso gratuito estandar";
        } else {
            return "Adulto mayor - Acceso gratuito preferencial";
        }
    } else if (tipo.equals("premium")) {
```

```
        if (edad < 12) {
            return "Nino - Acceso premium infantil";
        } else if (edad < 18) {
            return "Adolescente - Acceso premium completo";
        } else if (edad < 60) {
            return "Adulto - Acceso premium completo";
        } else {
            return "Adulto mayor - Acceso premium preferencial";
        }
    } else {
        return "ERROR: suscripcion no valida";
    }
}
```

Las pruebas se escribieron con JUnit 5. Cada prueba llama al método clasificar con una combinación de datos y compara el resultado con el valor esperado usando assertEquals. Se diseñaron casos en los límites de cada rango (11, 12, 17, 18, 59, 60) porque ahí es donde más fácil se cometen errores de programación.

4.1 Implementación en PSeInt

El mismo algoritmo se escribió en PSeInt, un programa que interpreta el pseudocódigo, lo ejecuta y genera el diagrama de flujo. El archivo se llama ClasificadorUsuarios.psc y se incluye en el repositorio. La Figura 3 muestra el código escrito en PSeInt.

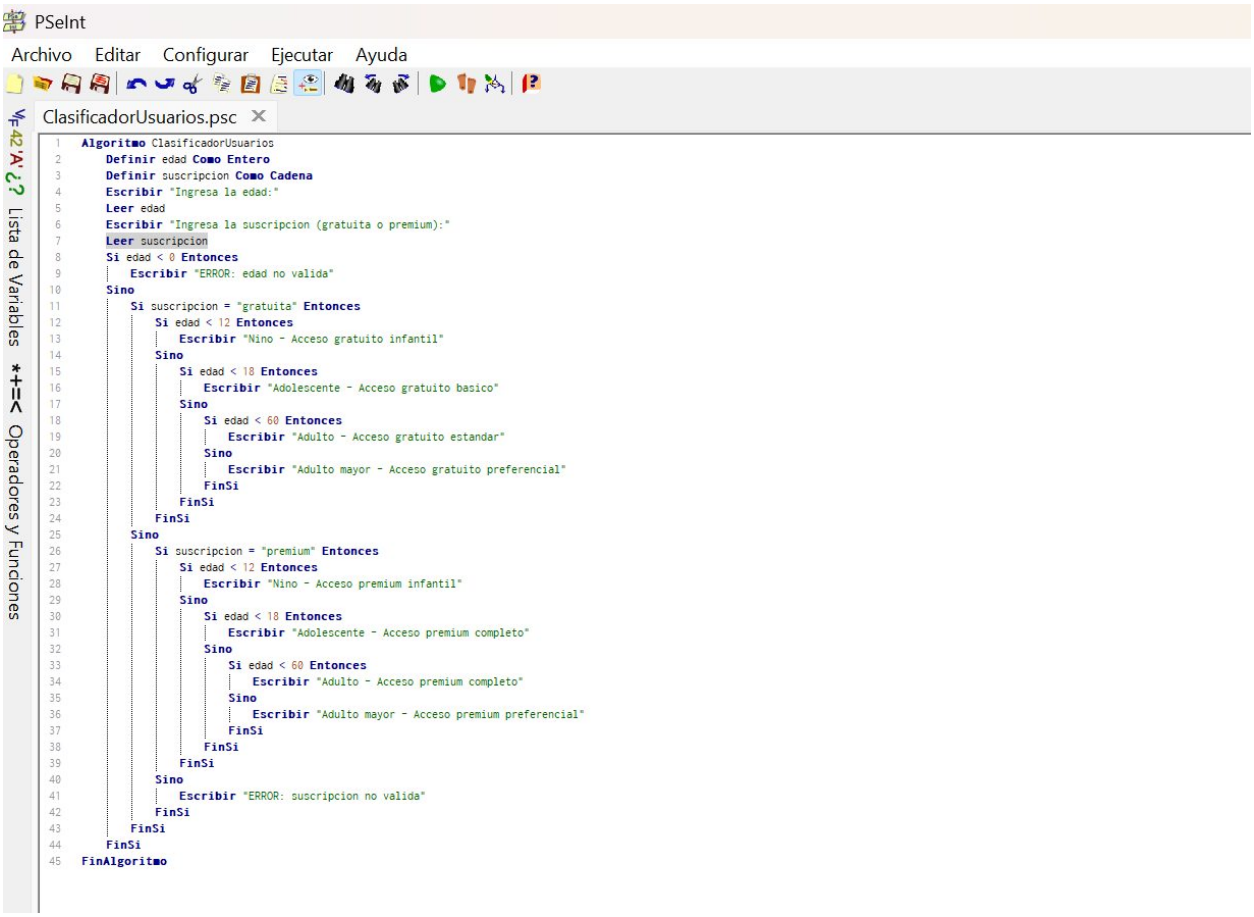


Figura 3. Pseudocódigo del clasificador escrito en PSeInt.

4.2 Implementación en Java

La misma lógica también se tradujo a Java, como se describe arriba. En PSeInt, las condicionales se escriben con Si, Sino y FinSi. En Java se usan if, else if y else. Ambas versiones tienen la misma estructura de decisiones y devuelven los mismos resultados.

Una diferencia importante en PSeInt es que compara los textos tal como se escriben. Por eso "Premium" con mayúscula o con espacios al final dan error. En Java, el método limpia los espacios y pasa el texto a minúsculas antes de comparar.

5. Resultados de las pruebas

La tabla siguiente muestra los 10 casos de prueba, sus entradas, el resultado esperado y el estado obtenido. Las pruebas se ejecutaron con un programa de consola que compara cada salida con el resultado esperado. Este programa usa la misma lógica del método clasificar.

Caso	Edad	Suscripcion	Resultado esperado	Estado
01	5	gratuita	Nino - Acceso gratuito infantil	Exitoso
02	11	premium	Nino - Acceso premium infantil (limite)	Exitoso
03	12	gratuita	Adolescente - Acceso gratuito basico (limite)	Exitoso
04	17	premium	Adolescente - Acceso premium completo	Exitoso
05	18	gratuita	Adulto - Acceso gratuito estandar (limite)	Exitoso
06	59	premium	Adulto - Acceso premium completo (limite)	Exitoso
07	60	gratuita	Adulto mayor - Acceso gratuito preferencial (limite)	Exitoso
08	75	premium	Adulto mayor - Acceso premium preferencial	Exitoso
09	-1	gratuita	ERROR: edad no valida	Exitoso
10	20	VIP	ERROR: suscripcion no valida	Exitoso

Resumen: de los 10 casos, 10 fueron exitosos y 0 fallidos. Todos los límites de edad (11, 12, 17, 18, 59 y 60) dieron la categoría correcta, y los dos casos de error devolvieron el mensaje esperado. Los resultados deben confirmarse en Eclipse con JUnit, porque el programa de consola no es el mismo entorno de la entrega.

6. Discusión

La solución cumple con los requisitos del reto. Clasifica correctamente a los usuarios en las cuatro categorías de edad, respeta los dos tipos de suscripción y detecta los datos inválidos. El uso de condicionales anidadas permitió revisar primero la validez de los datos y después la combinación de suscripción y edad, lo que hace que el flujo sea fácil de seguir.

Sin embargo, la solución tiene algunas limitaciones. La primera es que el bloque de suscripción premium repite casi el mismo código que el bloque gratuito. Si en el futuro se cambia un rango de edad, hay que cambiarlo en dos lugares, lo que aumenta la posibilidad de error. La segunda limitación es que la edad solo se revisa para que no sea negativa. Una edad de 300 años sería aceptada y clasificada como adulto mayor. La tercera es que los nombres de las categorías y los niveles de acceso están escritos directamente dentro del código, lo que hace difícil cambiarlos sin tocar la lógica.

Durante el diseño también se pensó en el orden de las preguntas. Se decidió revisar primero la edad y después la suscripción, pero el resultado sería el mismo si se hiciera al revés. Lo importante es que cada combinación tenga una sola respuesta. En la práctica, el orden sí afecta la lectura: al validar primero los datos, quien lea el código entiende de inmediato que los errores se atienden antes que cualquier regla de acceso. Esta forma de organizar el código ayuda a un programador nuevo a seguir la lógica sin perderse.

Otro punto importante es que las pruebas solo cubren los límites y algunos casos representativos. No prueban todas las combinaciones posibles de edad, pero sí cubren los lugares donde suele haber errores. Por eso se consideran suficientes para este nivel del curso.

7. Recomendaciones de mejora

Se proponen las siguientes mejoras para una futura versión del programa.

Primera: separar la clasificación por edad en su propio método. Así, el método clasificarEdad solo devolvería la categoría (niño, adolescente, adulto o adulto mayor), y otro método decidiría el nivel de acceso según la suscripción. Esto elimina la repetición de código y hace que cada parte sea más fácil de probar.

Segunda: usar un enum de Java para las categorías de edad y los tipos de suscripción. Un enum evita errores de escritura en los textos, por ejemplo escribir "Gratuita" con mayúscula en un lugar y sin mayúscula en otro.

Tercera: agregar una validación de edad máxima, por ejemplo que la edad no sea mayor a 120 años, y tratar los valores fuera de ese rango como error.

Cuarta: usar un switch para la categoría de edad, porque en algunos casos el código queda más claro que con muchos else if seguidos. Sin embargo, con rangos de números el if es más natural, por lo que esta mejora depende de cómo se organice el código.

Quinta: guardar los textos de los niveles de acceso en un archivo de configuración o en constantes, para poder cambiarlos sin modificar la lógica del programa.

8. Conclusiones

Este reto permitió practicar las estructuras de decisión anidadas en Java. Se aprendió que un problema con dos variables, como la edad y la suscripción, se puede resolver de forma ordenada si primero se validan los datos y luego se revisan las combinaciones. También se vio la importancia de probar los valores límite, porque son los que más errores producen.

El proyecto también mostró que escribir un buen pseudocódigo y un diagrama de flujo antes de programar facilita mucho la traducción al código. Aun así, la solución puede mejorar en organización y legibilidad, y las recomendaciones de este informe son el siguiente paso.

Bibliografía

Deitel, P., y Deitel, H. (2018). *Java: How to program, early objects* (11.a ed.). Pearson.

Oracle. (s. f.). *Java Platform, Standard Edition documentation*. https://docs.oracle.com/en/java/

JUnit Team. (s. f.). *JUnit 5 User Guide*. https://junit.org/junit5/docs/current/user-guide/

Nota: estas referencias se incluyen de forma general. Antes de entregar, verificar que los datos de autor, edición y enlace sean correctos.

Anexo A. Casos de prueba

Los casos de prueba se encuentran en la clase ClasificadorUsuariosTest del proyecto. Cada método tiene un nombre que indica el caso y el motivo de la prueba. La tabla de la sección 5 resume las entradas y los resultados esperados de cada uno.